Les pointeurs

Définition

Définition

Si x est un pointeur vers un entier, *x sera l'entier pointé par x. Ainsi si on écrit :

Exemple 1

```
int a=25;
int *x;
x=&a;
*x=25;
cout<<"*x="<<*x<<endl;
cout<<"x="<<x<<endl;
cout<<"&x="<<&x<<endl;
cout<<"a="<<a<<endl;
cout<<"&a="<<&a<<endl;
```

résultat

*x=.....
x=.....
&x=.....
a=.....
&a=.....

Définition

Il s'agit d'une notion importante du C++ mais qui est assez délicate à utiliser. Les pointeurs semblent au départ assez pratique à utiliser mais de nombreux bogues résultent de leur utilisation. Il convient donc de savoir les utiliser mais la prudence doit être de rigueur
Un pointeur est une variable qui contient l'adresse d'une autre variable du même type.

Déclaration

Déclaration d'un pointeur : int * x;
x est un pointeur vers un entier : x contient l'adresse en mémoire où est stocké un entier. Initialement le pointeur n'est pas initialisé : x vaut donc n'importe quelle adresse en RAM.
Si a est un entier, &a renvoie l'adresse réelle en mémoire de la variable a.

Exemple 1

```
int a;
int *x;
x=&a;
cout<<"*x="<<*x<<endl;
cout<<"x="<<x<<endl;
cout<<"&x="<<&x<<endl;
cout<<"a="<<a<<endl;
cout<<"&a="<<&a<<endl;
```

résultat

```
 *x=0
x=0x6ffe0c
&x=0x6ffe00
a=0
&a=0x6ffe0c
```


Exemple d'utilisation d'un pointeur

Exemple

```
#include<iostream>
using namespace std;
int main()
{
int a;
int *x,*y;
a=90;
x=&a;
cout << *x << endl;
*x=100;
cout << "a vaut : " << a << endl;
y=x;
*y=80;
cout << "a vaut : " << a << endl;
return 0;
}
```

résultat

```
*x=25
x=0x6ffe0c
&x=0x6ffe00
a=25
&a=0x6ffe0c
```

Explication

.....
.....
.....
.....
.....
.....

Passage par valeur et passage par adresse

Exemple

```
#include <iostream>
using namespace std;
#include<cstdlib>
void f1(int *x,int y)
{
      *x=1;
       y=2;
}
int main(int argc, char** argv) {

      int a=10,b=20;
      cout<<"avant le passage de la
fonction f1"<<endl;
      cout<<"a="<<a<<endl;
      cout<<"b="<<b<<endl;
      f1(&a,b);
      cout<<"après le passage de la
fonction f1"<<endl;
      cout<<"a="<<a<<endl;
      cout<<"b="<<b<<endl;
      return 0;
}
```

résultat

.....
.....
.....
.....
.....

Explication

.....
.....
.....
.....
.....
.....

L'instruction new

Exemple

```
#include<iostream>
using namespace std;
int main()
{
int *t;
int i;
t = new int[5];
if (t == NULL )
 cout << "pas assez de mémoire" << endl;
else
{
for ( i=0 ; i<5 ; i++ )
      t[i] = i * i;
for ( i=0 ; i<5 ; i++ )
      cout << t[i] << " ";
cout << endl;
for ( i=0 ; i<5 ; i++ )
      cout << (t+i) << " ";
cout << endl;
delete [] t;
for ( i=0 ; i<5 ; i++ )
      cout << t[i] << " ";
cout << endl;
for ( i=0 ; i<5 ; i++ )
      cout << (t+i) << " ";
cout << endl;
}
return 0;
}
```

résultat

.....
.....
.....
.....
.....

Explication

.....
.....
.....
.....
.....

Allocation dynamique

Définition

Un programme en C++ est capable de demander au système d'exploitation de la mémoire pour y stocker des données et de libérer ces zones mémoire au cours de l'exécution du programme : on appelle cela l'allocation dynamique de mémoire.

• Le programmeur est tenu de "rendre" au système d'exploitation, les zones mémoire qu'il aura réclamé dynamiquement, au plus tard lors de la fermeture du programme.

Pour effectuer de l'allocation dynamique de mémoire :on utilise les mots-clés new et delete

Syntaxe new

new type[taille];

new renvoie un tableau dont la taille est taille éléments, chaque élément étant de type type. S'il n'y a pas assez de mémoire, new renvoie NULL.

new renvoie un élément dont le type est type *.

Syntaxe delete

Syntaxe :

delete []t;

Si t est un tableau qui a été créé en utilisant la méthode précédente, l'utilisation de delete détruit ce tableau. Le tableau t n'est donc plus utilisable dès qu'on a utilisé l'opérateur delete.

5

6

L'instruction new

Exemple

```
#include<iostream>
using namespace std;
int main()
{
int *t;
int i;
t = new int[5];
if (t == NULL )
      cout << "pas assez de mémoire"
<< endl;
else
{
for ( i=0 ; i<5 ; i++ )
      t[i] = i * i;

for ( i=0 ; i<10 ; i++ )
      cout << t[i] << " ";
cout<<endl;
delete [] t;
}
return 0;
}
```

résultat

.....

.....

.....

.....

.....

Explication

.....

.....

.....

.....

.....

.....

7

8

Exemple

```
#include<iostream>
using namespace std;
int main()
{
int *t;
int i;
t= new int[5];
if (t == NULL )
   cout << "pas assez de mémoire" <<
endl;
else
{
for ( i=0 ; i<5 ; i++ )
t[i] = i * i;
for ( i=0 ; i<5 ; i++ )
cout << t[i] << " ";
cout << endl;
delete [] t;
t = new int[10];
if (t == NULL)
cout << "pas assez de mémoire" << endl;
else
{
for ( i=0 ; i<10 ; i++ )
t[i] = i * i;
for ( i=0 ; i<10 ; i++ )
cout << t[i] << " ";
cout << endl;
delete [] t;
}
}
return 0;
}
```

résultat

.....
.....
.....
.....
.....

Explication

.....
.....
.....
.....
.....
.....